

Disney+ Streaming Analytics Dashboard

Note: The data used in this analysis is synthetically generated for demonstration purposes.

1. Introduction

This tutorial demonstrates how to create an interactive dashboard for streaming media analytics using Python and Plotly. The dashboard combines multiple visualization types that work together to tell a cohesive story about viewer engagement patterns. The approach enables stakeholders to quickly identify regional differences, content preferences, and seasonal trends without requiring multiple separate reports.

1.1 Data Source Description

The data used in this analysis is synthetically generated to resemble streaming patterns similar to those found on major streaming platforms like Disney+. This synthetic data was created using Python and various libraries (pandas, numpy, random) to simulate realistic viewing behaviors while not containing any actual confidential information. The generation script ensures that the data follows realistic patterns and distributions of streaming behavior across different regions, content types, and time periods.

1.2 Dataset Structure

- Number of rows: ~4.5 million (representing daily viewing records)
- Number of columns: 6
- Time period: Full year 2023 (January 1 to December 31)

The dataset contains the following columns:

- `business_date` : The date when content was streamed (YYYY-MM-DD format)
- `account_id` : Unique 20-character identifier for each subscriber account
- `region` : Geographic region where content was viewed (Domestic, EMEA, APAC, LATAM)
- `content_type` : Type of content streamed ("Series" or "Film")
- `full_title` : Title of the content including season information for series
- `hours_streamed` : Number of hours the subscriber spent streaming that content on that day

2. Visualization Techniques & Library

This dashboard employs four complementary visualization types that work together to provide a complete understanding of streaming patterns:

1. **Bar Charts (HPS by Region)** - Provides an immediate snapshot of regional performance using the critical Hours Per Subscriber (HPS) metric, allowing for quick comparison between markets.
2. **Grouped Bar Charts (HPS by Content Type and Region)** - Extends the regional analysis by separating content types, revealing how different content categories perform across regions.
3. **Line Charts (Total Hours Streamed by Month)** - Displays temporal patterns that are impossible to see in snapshot views, enabling the identification of seasonal trends and year-over-year growth.

4. **Treemaps (Top Titles by Hours Streamed)** - Hierarchical visualization that efficiently shows both content type distribution and individual title performance in a space-efficient format.

These visualizations complement each other by addressing different analytical questions (what/where/when/how) while maintaining consistent color coding and metric definitions throughout.

2.1 Framework Selection: Plotly with Ipywidgets

This dashboard leverages **Plotly Express**, a high-level API for Plotly.js that makes it simple to create interactive visualizations with minimal code. Key benefits of this approach:

- **Open-source** - Plotly is freely available under the MIT license
- **Interactive by default** - All visualizations include hover information, zooming, and panning without additional code
- **Declarative syntax** - Clear mapping between data and visual elements makes code readable and maintainable
- **Jupyter integration** - Visualizations render directly in notebook cells

I've combined Plotly with **ipywidgets** to create control elements (date pickers, checkboxes) that allow for dynamic filtering of the underlying data. This approach provides dashboard interactivity without requiring a separate web server or JavaScript development.

3 Data Preparation

```
In [49]: # Standard data manipulation libraries
import pandas as pd
import numpy as np
from datetime import date

# Visualization libraries
import plotly.express as px
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns

# SQL functionality for pandas
from pandasql import sqldf

# Interactive widgets for dashboard
import ipywidgets as widgets
from IPython.display import display

# To display screenshot images in notebook
from IPython.display import Image, display, HTML

# Note: If libraries are missing, install with:
# pip install pandas numpy plotly matplotlib seaborn ipywidgets pandasql
```

```
In [ ]: # Create the dataframe
df = pd.read_csv('disney_plus_streaming_data.csv')
display(df.head(10))
```

	business_date	account_id	region	content_type	full_title	hours_streamed
0	2023-01-01	64QWFZ1DVLCTVWZ46GF5	Domestic	Series	Series_40: Legends of Zephra	0.23
1	2023-01-01	JDKBG4D14UC4TKTA5NUJ	Domestic	Series	Series_7: Legends of Zephra	0.23
2	2023-01-01	JDKBG4D14UC4TKTA5NUJ	Domestic	Series	Frozen Tales Series	5.00
3	2023-01-01	J0XBVXCIJD4862SPA64Y	Domestic	Series	Series_21: Knights of Eldoria	0.23
4	2023-01-01	V1QGG102U0DXMPF7N4YM	Domestic	Series	Marvel Heroes United	5.00
5	2023-01-01	MO3VN1G7GDWKPC4HWXD8	Domestic	Series	Moana: Island Chronicles	5.00
6	2023-01-01	MO3VN1G7GDWKPC4HWXD8	Domestic	Series	Bluey Jr.	5.00
7	2023-01-01	TOAN83YHACFOL2493Z0M	Domestic	Series	Series_50: Legends of Zephra	0.20
8	2023-01-01	TOAN83YHACFOL2493Z0M	Domestic	Film	Finding Marlin	3.09
9	2023-01-01	WY4DVYKYVUA1QK8KCD44	Domestic	Film	The Little Robot	3.48

```
In [3]: # Review each column's data type
display(df.info(10))

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4515998 entries, 0 to 4515997
Data columns (total 6 columns):
#   Column      Dtype
---  -
0   business_date  object
1   account_id    object
2   region        object
3   content_type  object
4   full_title    object
5   hours_streamed float64
dtypes: float64(1), object(5)
memory usage: 206.7+ MB
None
```

```
In [4]: # Convert 'business_date' column to datetime
df['business_date'] = pd.to_datetime(df['business_date'])
```

```
In [5]: # Confirm each column's data type
display(df.info(10))

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4515998 entries, 0 to 4515997
Data columns (total 6 columns):
#   Column      Dtype
---  -
0   business_date  datetime64[ns]
1   account_id    object
2   region        object
3   content_type  object
4   full_title    object
5   hours_streamed float64
dtypes: datetime64[ns](1), float64(1), object(4)
memory usage: 206.7+ MB
None
```

```
In [6]: # display(df.head(10))
display(df.iloc[100000:110000])
```

	business_date	account_id	region	content_type	full_title	hours_streamed
100000	2023-01-11	R8OCKIENVCGD24SEP9MU	LATAM	Series	Marvel Heroes United	5.00
100001	2023-01-11	R8OCKIENVCGD24SEP9MU	LATAM	Series	Series_40: Legends of Zephra	0.35
100002	2023-01-11	REEC02M9A08AAKTYVQFQ	LATAM	Film	Moana's Journey	5.00
100003	2023-01-11	MLGBFA2BXD8S3U1NCMKV	LATAM	Series	Star Blasters	5.00
100004	2023-01-11	QJAB9NE84HKAFDMW35UT	LATAM	Series	Series_50: Legends of Zephra	0.38
...	...	...	...	...	...	...
109995	2023-01-13	1LJ22YQF8XI40KK87CCE	Domestic	Series	Series_22: Chronicles of Aralon	0.23
109996	2023-01-13	8PINX3OOGO7WST9DLCKB	Domestic	Series	Frozen Tales Series	5.00
109997	2023-01-13	FO14CGBMDMG8GCACS3GU	Domestic	Series	Bluey Jr.	5.00
109998	2023-01-13	39EL9J4AFIG5WFKSLVTC	Domestic	Series	Bluey Jr.	5.00
109999	2023-01-13	RCB1DKLBMLN5W9LKHC2F	Domestic	Series	Bluey Jr.	5.00

10000 rows × 6 columns

4. Analysis Datasets

4.1 Dataset 1: Average Weekly Hours Per Subscriber (HPS) by Region

```
In [7]: # Define a helper function to make running SQL queries easier
# Helper function: sql_query is a wrapper function that passes your SQL query to sqldf.
# The globals() parameter makes all variables in our global namespace (including the DataFrame df) available to the SQL query
def sql_query(query):
    return sqldf(query, globals())

# Dataset 1: Query weekly HPS by Region
weekly_hps_by_region = sql_query("""
-- Calc rolling weekly hours by account and group by region
WITH l7_hours AS (
    SELECT
        DATE(business_date) as business_date, -- This converts to YYYY-MM-DD format and removes timestamp
        account_id,
        region,
        SUM(hours_streamed) OVER(PARTITION BY account_id ORDER BY business_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) as hours_l7,
        COUNT(*) OVER(PARTITION BY account_id ORDER BY business_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) as day_count
    FROM df
    GROUP BY business_date, account_id, region
)
SELECT
    business_date,
    region,
    -- Calculate average weekly hours per subscriber (HPS)
    SUM(hours_l7) / COUNT(DISTINCT account_id) as avg_weekly_hps
FROM l7_hours
WHERE day_count = 7 -- Only include complete 7-day windows (i.e., filter weeks with partial weeks)
GROUP BY business_date, region
ORDER BY business_date asc
""")
```

```
In [8]: # View Output
display(weekly_hps_by_region.iloc[0:10])
```

	business_date	region	avg_weekly_hps
0	2023-01-12	LATAM	21.230000
1	2023-01-13	APAC	4.630000
2	2023-01-15	APAC	1.850000
3	2023-01-15	Domestic	17.460000
4	2023-01-15	EMEA	17.160000
5	2023-01-15	LATAM	25.785000
6	2023-01-16	EMEA	10.380000
7	2023-01-16	LATAM	16.556667
8	2023-01-17	Domestic	13.640000
9	2023-01-17	EMEA	10.655000

4.2 Dataset 2: HPS by Region and Content Type

```
In [9]: # Dataset 2: Query weekly HPS by Region and Content Type
weekly_hps_by_content_type = sql_query("""
-- Calc rolling weekly hours by account and group by region
```

```

WITH l7_hours AS (
    SELECT
        DATE(business_date) as business_date,  -- This converts to YYYY-MM-DD format and removes timestamp
        account_id,
        region,
        content_type,
        SUM(hours_streamed) OVER(PARTITION BY account_id ORDER BY business_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) as hours_l7,
        COUNT(*) OVER(PARTITION BY account_id ORDER BY business_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) as day_count
    FROM df
    GROUP BY business_date, account_id, region
)
SELECT
    business_date,
    region,
    content_type,
    -- Calculate average weekly hours per subscriber (HPS)
    SUM(hours_l7) / COUNT(DISTINCT account_id) as avg_weekly_hps
FROM l7_hours
WHERE day_count = 7 -- Only include complete 7-day windows (i.e., filter weeks with partial weeks)
GROUP BY business_date, region, content_type
ORDER BY business_date asc
""")

```

```

In [10]: # View Ouput
display(weekly_hps_by_content_type.iloc[0:10])

```

	business_date	region	content_type	avg_weekly_hps
0	2023-01-12	LATAM	Film	21.23
1	2023-01-13	APAC	Film	4.63
2	2023-01-15	APAC	Series	1.85
3	2023-01-15	Domestic	Series	17.46
4	2023-01-15	EMEA	Series	17.16
5	2023-01-15	LATAM	Film	30.33
6	2023-01-15	LATAM	Series	21.24
7	2023-01-16	EMEA	Film	10.38
8	2023-01-16	LATAM	Film	25.71
9	2023-01-16	LATAM	Series	11.98

4.3 Dataset 3: Hours Streamed by Title

```

In [11]: # Dataset 3: Query daily Hours Streamed by Title by Region
hours_streamed_by_title = sql_query("""
-- Calc rolling weekly hours by account and group by region
SELECT
    DATE(business_date) as business_date,  -- This converts to YYYY-MM-DD format and removes timestamp
    full_title,
    content_type,
    region,
    SUM(hours_streamed) as hours_streamed
FROM df
GROUP BY business_date,full_title, content_type, region
""")

```

```
ORDER BY business_date asc
)
)
)
```

```
In [12]: # View Output
display(hours_streamed_by_title.iloc[0:10])
```

	business_date	full_title	content_type	region	hours_streamed
0	2023-01-01	Aladdin: The New Adventure	Film	APAC	20.54
1	2023-01-01	Aladdin: The New Adventure	Film	Domestic	116.41
2	2023-01-01	Aladdin: The New Adventure	Film	EMEA	50.73
3	2023-01-01	Aladdin: The New Adventure	Film	LATAM	105.00
4	2023-01-01	Avatar 3	Film	APAC	18.44
5	2023-01-01	Avatar 3	Film	Domestic	144.89
6	2023-01-01	Avatar 3	Film	EMEA	39.73
7	2023-01-01	Avatar 3	Film	LATAM	130.00
8	2023-01-01	Beauty and the Dragon	Film	APAC	18.53
9	2023-01-01	Beauty and the Dragon	Film	Domestic	108.24

5. Dashboard Component Development

The dashboard consists of four visualization components built using Plotly Express. Each component is designed as a standalone function that:

1. Accepts filtered data based on user selections
2. Creates an appropriately configured visualization
3. Returns a Plotly figure object

5.1 Regional HPS Snapshot

```
In [76]: # 1. HPS by Region Bar Chart
def create_region_hps_snapshot(weekly_hps_data, date_str, selected_regions):
    """
    Creates a bar chart showing Hours Per Subscriber (HPS) by Region for a specific date.

    Parameters:
    -----
    weekly_hps_data : pandas DataFrame
        DataFrame with weekly HPS data
    date_str : str
        Date to create the snapshot for (YYYY-MM-DD format)
    selected_regions : list
        List of regions to include in the visualization

    Returns:
    -----
    plotly figure object
        The bar chart visualization
    """
    # Filter data for the specified date and regions
```

```

filtered_df = weekly_hps_data[
    (weekly_hps_data['business_date'] == date_str) &
    (weekly_hps_data['region'].isin(selected_regions))
]

# Create the bar chart
fig = px.bar(
    filtered_df,
    x='region',          # X-axis shows regions
    y='avg_weekly_hps',  # Y-axis shows the HPS metric
    color='region',      # Color bars by region
    title=f'Weekly HPS by Region ({date_str})'
)

# Customize layout for better appearance
fig.update_layout(
    yaxis_title='Average Weekly HPS',
    xaxis_title='Region',
    legend_title = 'Region',
    height=400,
)

return fig

## Interactive Plot
create_region_hps_snapshot(weekly_hps_by_region, '2023-07-17',['APAC', 'Domestic', 'EMEA', 'LATAM'])

## Plot Reference Image
display(Image('assets/region_hps_snapshot.png', width=1200)) # Control the width

```



5.2 Content Type Comparison

```

In [73]: # 2. HPS by Content Type and Region (Grouped Bar Chart)
def create_content_type_hps_snapshot(weekly_content_type_data, date_str, selected_regions):
    """
    Creates a grouped bar chart showing HPS by Content Type and Region.

    Parameters:
    -----
    weekly_content_type_data : pandas DataFrame
        DataFrame with weekly HPS data by content type
    date_str : str

```



```

        Date to create the snapshot for (YYYY-MM-DD format)
selected_regions : list
    List of regions to include in the visualization

Returns:
-----
plotly figure object
    The grouped bar chart visualization
"""
# Filter data for the specified date and regions
filtered_df = weekly_content_type_data[
    (weekly_content_type_data['business_date'] == date_str) &
    (weekly_content_type_data['region'].isin(selected_regions))
]

# Create the grouped bar chart
fig = px.bar(
    filtered_df,
    x='region',          # X-axis shows regions
    y='avg_weekly_hps',  # Y-axis shows the HPS metric
    color='content_type', # Group and color by content type
    barmode='group',     # Display bars side-by-side instead of stacked
    title=f'HPS by Content Type and Region ({date_str})'
)

# Customize layout
fig.update_layout(
    yaxis_title='Average Weekly HPS',
    xaxis_title='Region',
    legend_title='Content Type',
    height=400,
)

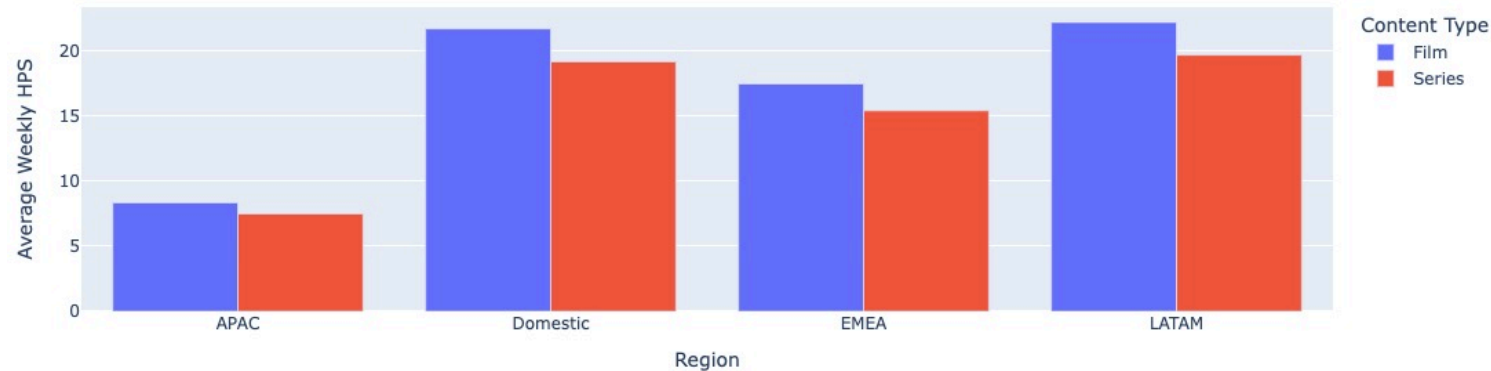
return fig

## Interactive Plot
# create_content_type_hps_snapshot(weekly_hps_by_content_type, '2023-07-17',['APAC', 'Domestic', 'EMEA', 'LATAM'])

## Plot Reference Image
display(Image('assets/content_type_hps_snapshot.jpeg', width=1000)) # Control the width

```

HPS by Content Type and Region (2023-07-17)



5.3 Monthly Trends Analysis

```
In [72]: # 3. Hours Streamed by Month (Line Chart)
def create_monthly_hours_chart(hours_by_title_data, start_date, end_date, selected_regions):
    """
    Creates a line chart showing total hours streamed by month.

    Parameters:
    -----
    hours_by_title_data : pandas DataFrame
        DataFrame with hours streamed by title
    start_date : datetime
        Start date for filtering data
    end_date : datetime
        End date for filtering data
    selected_regions : list
        List of regions to include in the visualization

    Returns:
    -----
    plotly figure object
        The line chart visualization
    """
    # Convert dates to datetime for comparison
    hours_by_title_dates = pd.to_datetime(hours_by_title_data['business_date'])

    # Create date filter mask
    date_mask = (hours_by_title_dates >= start_date) & (hours_by_title_dates <= end_date)

    # Filter data by date range and selected regions
    filtered_df = hours_by_title_data[
        date_mask & hours_by_title_data['region'].isin(selected_regions)
    ].copy()

    # Convert dates to month format (YYYY-MM)
    filtered_df['month'] = pd.to_datetime(filtered_df['business_date']).dt.strftime('%Y-%m')

    # Group by month and sum the hours streamed
    monthly_hours = filtered_df.groupby('month')['hours_streamed'].sum().reset_index()
```

```

# Create the line chart
fig = px.line(
    monthly_hours,
    x='month',          # X-axis shows months
    y='hours_streamed', # Y-axis shows total hours
    markers=True,       # Show markers at each data point
    title='Total Hours Streamed by Month'
)

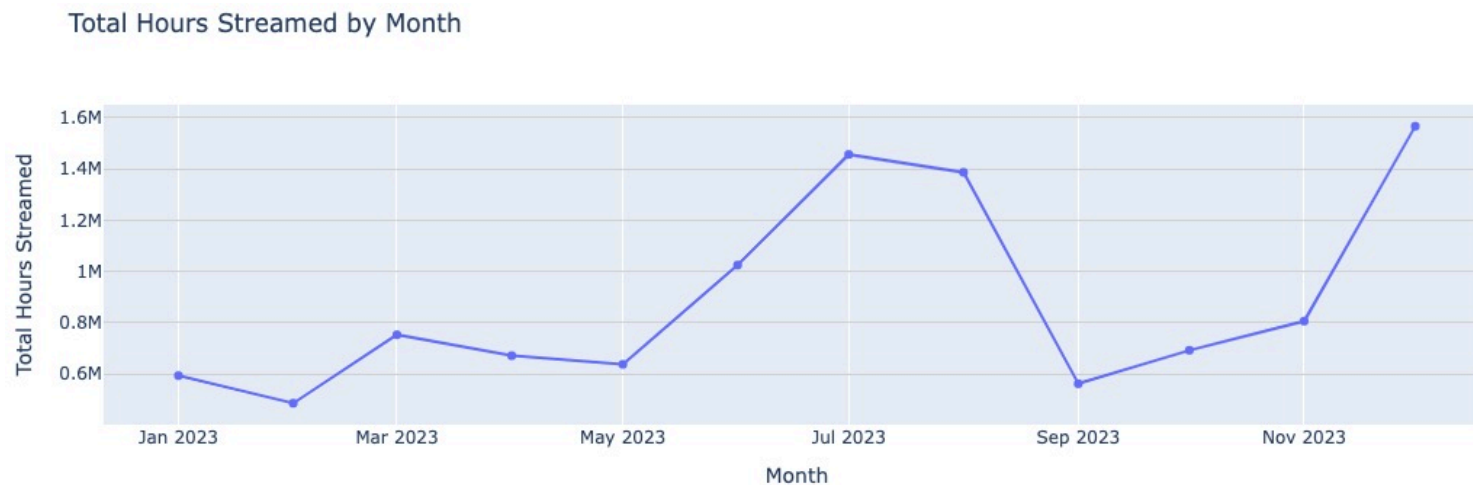
# Add thousands formatter to y-axis values
fig.update_layout(
    yaxis_title='Total Hours Streamed',
    xaxis_title='Month',
    height=400,
    # Add gridlines for better readability
    yaxis=dict(
        gridcolor='lightgray'
    )
)

return fig

## Interactive Plot
# create_monthly_hours_chart(hours_streamed_by_title, '2023-01-01', '2023-12-31', ['APAC', 'Domestic', 'EMEA', 'LATAM'])

## Plot Reference Image
display(Image('assets/monthly_hours_chart.jpeg', width=1000)) # Control the width

```



5.4 Top Titles Visualization

```

In [74]: # 4. Top Titles Treemap
def create_top_titles_treemap(hours_by_title_data, start_date, end_date, selected_regions, top_n=20):
    """
    Creates a treemap showing top titles by total hours streamed.

    Parameters:
    -----

```

```

hours_by_title_data : pandas DataFrame
    DataFrame with hours streamed by title
start_date : datetime
    Start date for filtering data
end_date : datetime
    End date for filtering data
selected_regions : list
    List of regions to include in the visualization
top_n : int
    Number of top titles to display (default: 20)

Returns:
-----
plotly figure object
    The treemap visualization
"""
# Convert dates to datetime for comparison
hours_by_title_dates = pd.to_datetime(hours_by_title_data['business_date'])

# Create date filter mask
date_mask = (hours_by_title_dates >= start_date) & (hours_by_title_dates <= end_date)

# Filter data by date range and selected regions
filtered_df = hours_by_title_data[
    date_mask & hours_by_title_data['region'].isin(selected_regions)
].copy()

# Calculate total hours streamed for each title
title_hours = filtered_df.groupby('full_title')['hours_streamed'].sum().reset_index()

# Add content type information for color coding
title_metadata = filtered_df[['full_title', 'content_type']].drop_duplicates()
title_data = title_hours.merge(title_metadata, on='full_title')

# Sort by hours and get the top N titles
top_titles = title_data.sort_values('hours_streamed', ascending=False).head(top_n)

# Create the treemap visualization
fig = px.treemap(
    top_titles,
    path=['content_type', 'full_title'], # Hierarchy: content_type -> title
    values='hours_streamed',             # Size boxes by hours streamed
    color='content_type',                 # Color by content type
    title=f'Top {top_n} Titles by Total Hours Streamed',
    hover_data=['hours_streamed']         # Show hours streamed in tooltip
)

# Customize layout and tooltip format
fig.update_layout(
    height=450,
    margin=dict(t=50, l=25, r=25, b=25) # Adjust margins
)

# Format the hover tooltip to show hours with comma separators
fig.update_traces(
    hovertemplate='<b>{%label}</b><br>Hours: {%value:,.0f}<extra></extra>'
)

return fig

```

```
## Interactive Plot
# create_top_titles_treemap(hours_streamed_by_title, '2023-01-01', '2023-12-31', ['APAC', 'Domestic', 'EMEA', 'LATAM'], 20)

## Plot Reference Image
display(Image('assets/top_titles_treemap.jpeg', width=1000)) # Control the width
```

Top 20 Titles by Total Hours Streamed



6. Dashboard Integration

Now that we've created individual visualization components, we'll combine them into a cohesive interactive dashboard. The dashboard will include:

- **Filter Controls:** Date range selection and region checkboxes
- **Layout:** 2x2 grid of visualizations
- **Interactivity:** Update button to refresh all visualizations based on selected filters

```
In [45]: def create_streaming_dashboard(weekly_hps_by_region, weekly_hps_by_content_type, hours_streamed_by_title):
        """
        Creates an interactive dashboard with four visualization types:
        1. HPS by Region (Bar Chart)
        2. HPS by Content Type and Region (Grouped Bar Chart)
        3. Hours Streamed by Month (Line Chart)
        4. Top Titles (Treemap)
        """
        # Create output containers for the four plots
        plot1 = widgets.Output()
        plot2 = widgets.Output()
        plot3 = widgets.Output()
        plot4 = widgets.Output()

        # Get dates and regions for filters
        all_dates = sorted(pd.to_datetime(weekly_hps_by_region['business_date']).unique())
        all_regions = sorted(weekly_hps_by_region['region'].unique())

        # Date range selector setup
```

```

min_date = all_dates[0]
max_date = all_dates[-1]

# Convert to Python date objects for DatePicker
min_date_py = date(min_date.year, min_date.month, min_date.day)
max_date_py = date(max_date.year, max_date.month, max_date.day)

# Create date picker widgets
start_date_picker = widgets.DatePicker(
    description='Start:',
    value=min_date_py,
    disabled=False
)

end_date_picker = widgets.DatePicker(
    description='End:',
    value=max_date_py,
    disabled=False
)

# Region filter using checkboxes
region_checkboxes = {region: widgets.Checkbox(
    value=True,
    description=region,
    disabled=False,
    indent=False
) for region in all_regions}

# Create update button
update_button = widgets.Button(
    description='Update Dashboard',
    button_style='primary',
    layout=widgets.Layout(width='auto')
)

# Update function that refreshes all charts
def update_dashboard(b):
    # Get selected values from widgets
    selected_start_date = pd.Timestamp(start_date_picker.value)
    selected_end_date = pd.Timestamp(end_date_picker.value)
    selected_regions = [region for region, checkbox in region_checkboxes.items() if checkbox.value]

    # Default to all regions if none selected
    if not selected_regions:
        selected_regions = all_regions

    # Clear all plot areas
    for p in [plot1, plot2, plot3, plot4]:
        p.clear_output(wait=True)

    # Find latest date in selected range for snapshot views
    weekly_hps_by_region_dates = pd.to_datetime(weekly_hps_by_region['business_date'])
    date_mask = (weekly_hps_by_region_dates >= selected_start_date) & \
        (weekly_hps_by_region_dates <= selected_end_date)

    filtered_dates = weekly_hps_by_region[date_mask]['business_date']
    if not filtered_dates.empty:
        latest_date = pd.to_datetime(filtered_dates).max()
    else:
        latest_date = max_date # Use the latest available date if filtered set is empty

```

```

# Format as string for querying
latest_date_str = latest_date.strftime('%Y-%m-%d')

# Generate and display each visualization
with plot1:
    fig1 = create_region_hps_snapshot(weekly_hps_by_region, latest_date_str, selected_regions)
    fig1.show()

with plot2:
    fig2 = create_content_type_hps_snapshot(weekly_hps_by_content_type, latest_date_str, selected_regions)
    fig2.show()

with plot3:
    fig3 = create_monthly_hours_chart(hours_streamed_by_title, selected_start_date,
                                      selected_end_date, selected_regions)
    fig3.show()

with plot4:
    fig4 = create_top_titles_treemap(hours_streamed_by_title, selected_start_date,
                                    selected_end_date, selected_regions, 20)
    fig4.show()

# Set up button click handler
update_button.on_click(update_dashboard)

# Create the dashboard layout components
title = widgets.HTML("<h1 style='text-align:left'>Disney+ Streaming Analytics Dashboard</h1>")

# Date range section
date_range_box = widgets.HBox([
    widgets.VBox([
        widgets.Label('Date Range:'),
        widgets.HBox([start_date_picker, end_date_picker])
    ])
])

# Region selection section
region_label = widgets.HTML("<b>Select Region(s):</b>")
region_box = widgets.VBox([
    region_label,
    widgets.HBox([region_checkboxes[region] for region in all_regions])
])

# Controls layout
filters_left = widgets.VBox([date_range_box, region_box])
update_button_container = widgets.VBox([
    widgets.Label(" "), update_button],
    layout=widgets.Layout(justify_content='flex-end', align_items='flex-end')
)

# Combine filter controls
filters = widgets.HBox([
    filters_left,
    update_button_container
], layout=widgets.Layout(justify_content='space-between'))

# Layout for plots in a 2x2 grid
top_row = widgets.HBox([plot1, plot2])
bottom_row = widgets.HBox([plot3, plot4])

```

```

# Build complete dashboard
dashboard = widgets.VBox([
    title,
    widgets.Box([filters], layout=widgets.Layout(
        border='1px solid #ddd',
        padding='10px',
        margin='5px',
        border_radius='5px'
    )),
    top_row,
    bottom_row
])

# Display the dashboard
display(dashboard)

# Initialize dashboard with default settings
update_dashboard(None)

```

```

In [ ]: ## Interactive Dashboard
# create_streaming_dashboard(weekly_hps_by_region, weekly_hps_by_content_type, hours_streamed_by_title)

```

```

In [ ]: ## Dashboard Reference Image
display(Image('assets/dashboard.jpeg', width=1600)) # Control the width

```


Disney+ Streaming Analytics Dashboard

Date Range:

Start: 01/01/2023

End: 12/31/2023

Select Region(s):

☒ APAC

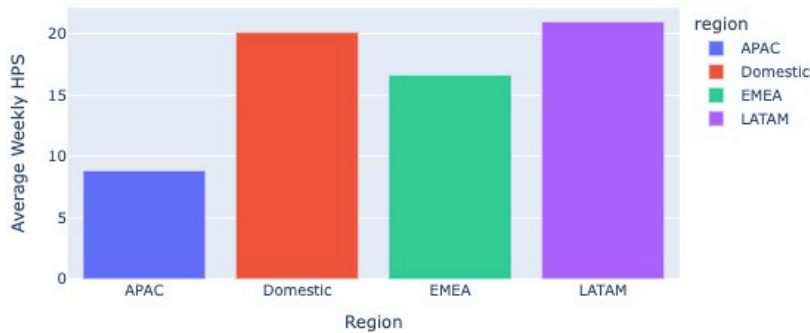
☒ Domestic

☒ EMEA

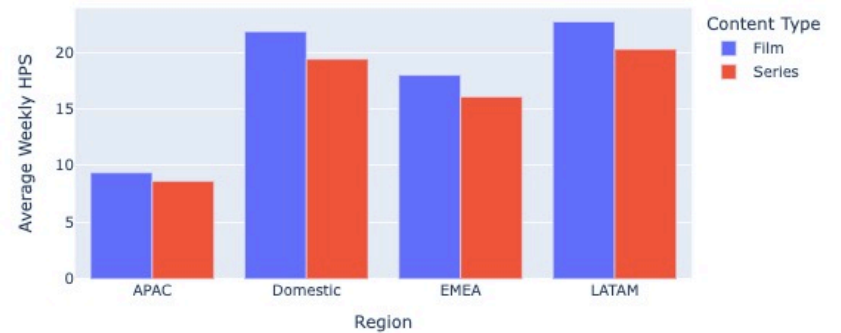
☒ LATAM

Update Dashboard

Weekly HPS by Region (2023-12-31)



HPS by Content Type and Region (2023-12-31)



Total Hours Streamed by Month



Top 20 Titles by Total Hours Streamed



7. Results & Insights

7.1 Technical Summary

This interactive dashboard combines four complementary visualization types to create a comprehensive analysis system using Plotly and ipywidgets:

- Bar charts for regional performance comparison
- Grouped bar charts for content type analysis
- Line charts for temporal trends
- Treemaps for hierarchical content performance

Key implementation features include modular visualization functions, consistent styling, and interactive filtering controls that allow stakeholders to explore patterns dynamically rather than through static reports.

Potential extensions could include geographic mapping, automated statistical analysis, or deployment as a standalone web application.

7.2 Business Insights & Recommendations

Key Insights

Regional Engagement

- **LATAM** and **Domestic** markets show highest engagement at **21** and **20** average weekly HPS respectively
- **APAC** shows significantly lower engagement compared to other regions (less than half of LATAM)

Content Type Performance

- **Film content** consistently outperforms series engagement across all regions by approximately **1.2x**
- This content type advantage is most pronounced in **LATAM** and **Domestic** markets

Seasonal Patterns

- Strong summer engagement peaks during **June, July, and August**
- Second significant engagement spike in **December** around holiday season
- **January** and **September** represent the lowest engagement periods

Content Analysis

- Series remain important despite lower overall engagement rates
- Top series performers include **Marvel Heroes United, Bluey Jr., and Star Blasters**
- Film category shows more diversified viewership across multiple titles

Strategic Recommendations

1. **Counter Post-Summer Drop:** Implement targeted promotion campaign in September to mitigate the engagement decline after summer viewing peaks
2. **Content Investment Rebalancing:**
 - Maintain film investment as our engagement foundation
 - Increase investment in high-performing series franchises to build sustained engagement
3. **Regional Focus:**
 - Investigate causes of low APAC engagement and develop region-specific content strategy
 - Leverage the success patterns from LATAM to other markets where applicable

4. Seasonal Planning:

- Align major content releases with natural engagement dips in January and September
- Enhance content offerings during peak viewing periods to maximize engagement

Analysis based on synthetic Disney+ streaming data covering the full 2023 calendar year across all global regions.